TRAVELSYNC

Intelligent Travel Planning & Information Platform

DSA Project | Java | Review-1 | CO1–CO3

TEAM MEMBERS

1. Y. Keerthika
2. N. Jyothirmayee
3. M. Mallika

GUIDE

M. Syamala Mam

SUBJECT CODE

25CS2103E

Problem Statement

Travel information is spread across many sources. Users misspell destination names, struggle to compare descriptions, arrange multi-stop routes and look for places that fit their preferences.

TravelSync applies string matching, sequence alignment and dynamic programming to these tasks inside a web application.

- **Finding travel information**
- **Misspelled destination queries**
- **Comparing descriptions**
- **Arranging multi-stop routes**
- **Matching user preferences**

Travel Data → **Search, Comparison & Planning Problems** → **DSA-Based Solutions**

Why TravelSync?

Destination Discovery

Explore and search destinations with live travel information.

Typo Suggestions

Suggest destinations when a query is misspelled.

Description Comparison

Find shared and similar text between descriptions.

Itinerary Optimization

Order multi-stop routes with Bitmask DP.

Activity Planning

Choose non-overlapping activities with Interval DP.

Personalized Recommendations

Rank destinations by preference with SOS DP.

Project Objectives

01



Build a modular web application (React + Spring Boot)

02



Apply string-search and matching algorithms

03



Compare travel descriptions using text algorithms

04



Optimize travel plans with dynamic programming

05



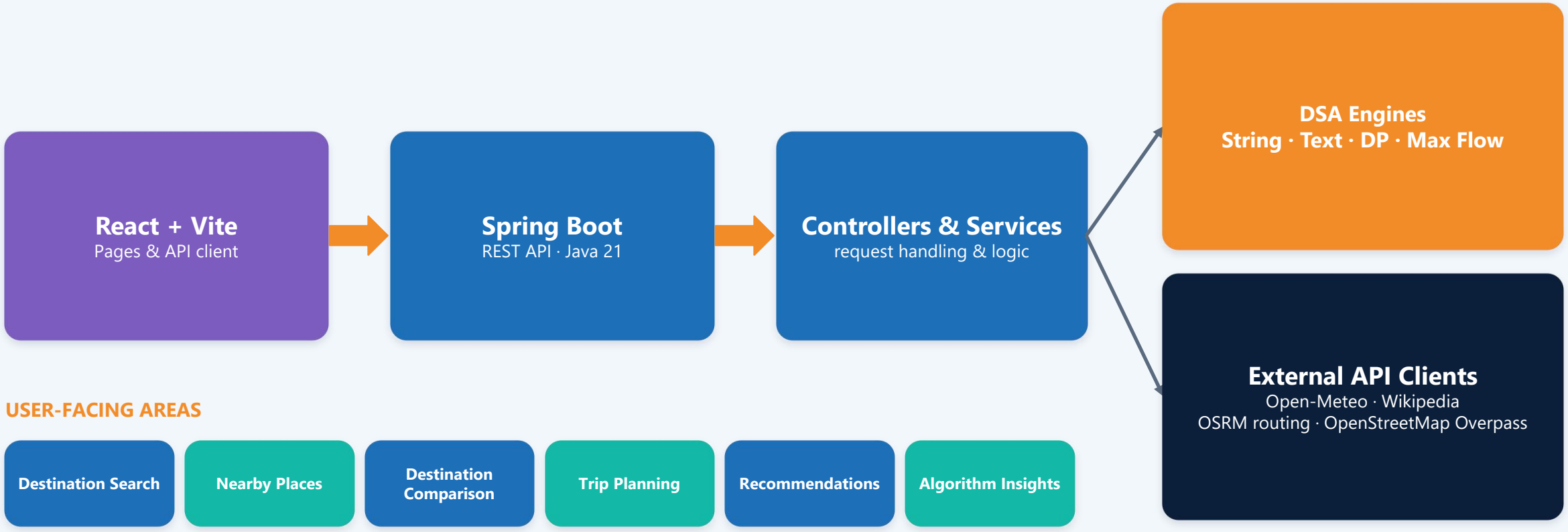
Integrate external travel-information and routing services

06



Test the algorithm implementations

System Overview

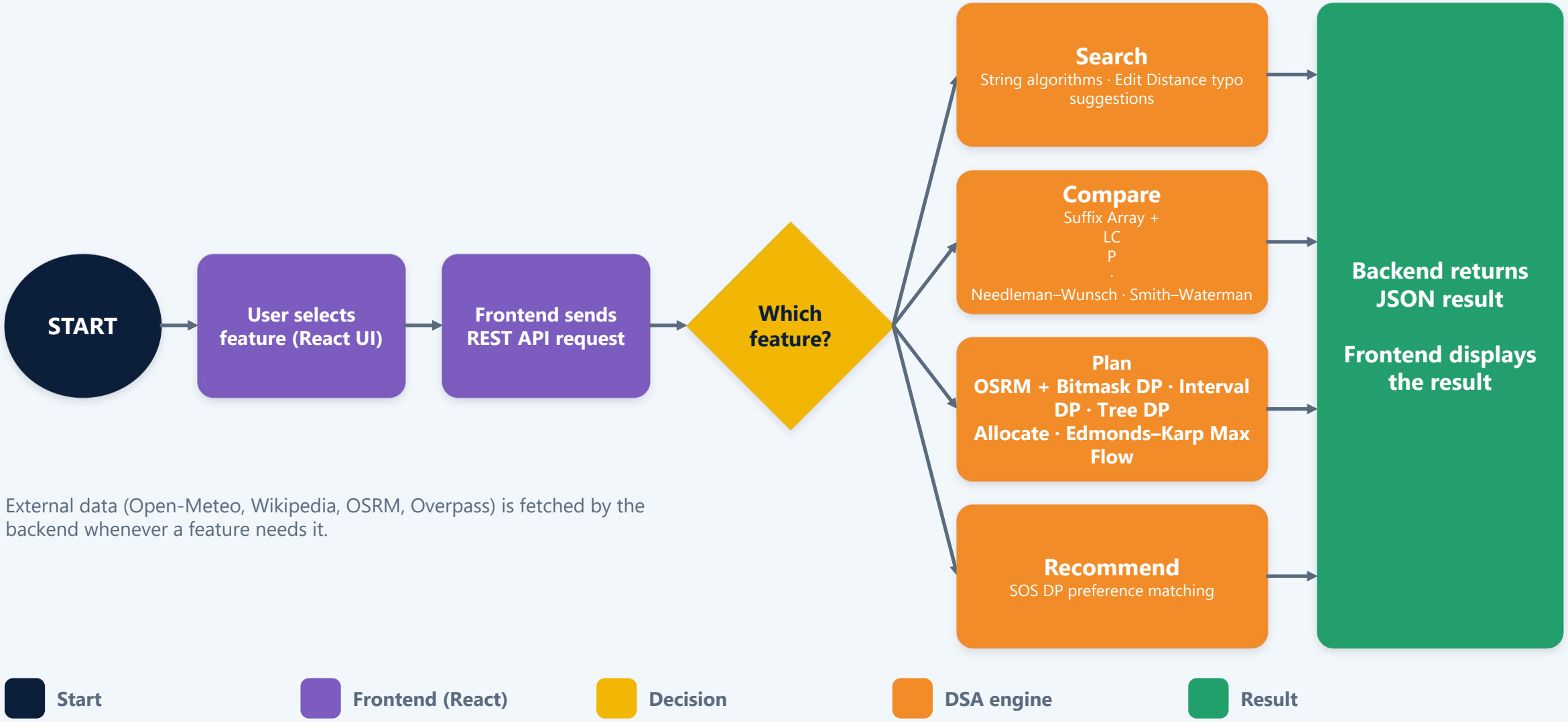


USER-FACING AREAS

- Destination Search
- Nearby Places
- Destination Comparison
- Trip Planning
- Recommendations
- Algorithm Insights

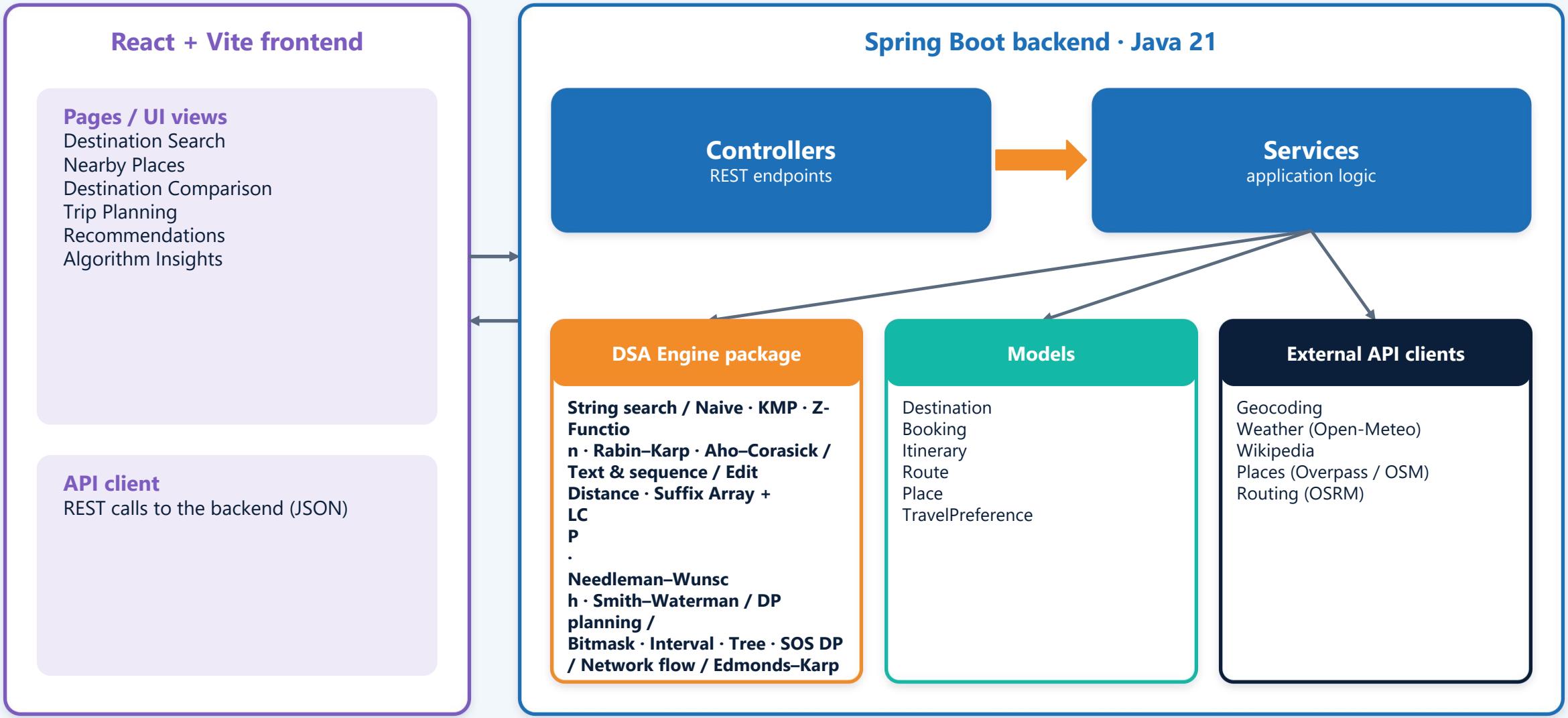
Results are returned as JSON to the React frontend and displayed to the user.

System Workflow



External data (Open-Meteo, Wikipedia, OSRM, Overpass) is fetched by the backend whenever a feature needs it.

Application Architecture



Data Structures & Algorithms

STRING SEARCH & MATCHING

- Naïve String Search
- KMP
- Z-Function
- Rabin–Karp
- Aho–Corasick

TEXT COMPARISON & ALIGNMENT

- Edit Distance
- Suffix Array
- LCP (Kasai)
- Needleman–Wunsch
- Smith–Waterman

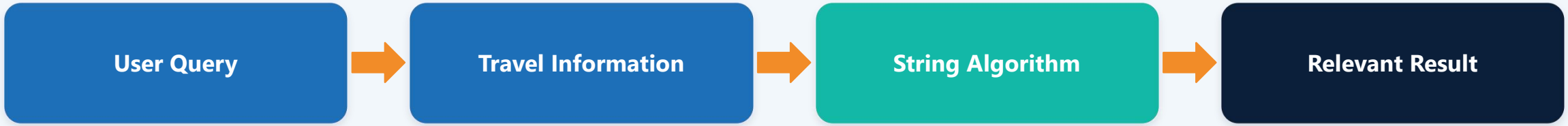
DYNAMIC PROGRAMMING (PLANNING)

- Bitmask DP
- Interval DP
- Tree DP
- SOS DP

NETWORK FLOW
Edmonds–Karp Max Flow

15 algorithm engines are implemented: the 14 shown above plus Edmonds–Karp maximum flow for activity allocation. Edit Distance and both sequence-alignment algorithms also use dynamic programming.

String Search in TravelSync



Naïve Search

Baseline string matching

KMP & Z-Function

Pattern searching in travel text

Aho–Corasick

Multiple keyword search

Rabin–Karp

Booking-code search

Edit Distance

Destination typo suggestions

Travel Description Comparison

SUFFIX ARRAY + LCP

Finds the longest common substring shared by two destination descriptions.

NEEDLEMAN–WUNSCH

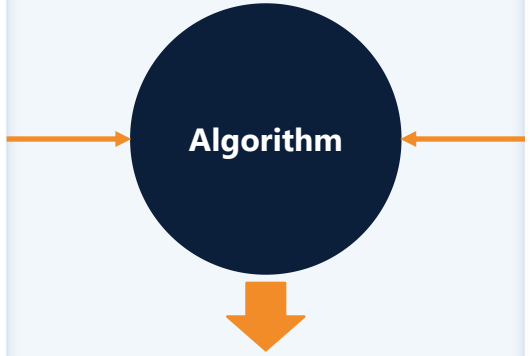
Aligns two complete descriptions end to end and scores their similarity.

SMITH–WATERMAN

Finds the best-matching local region between two descriptions.

DESCRIPTION A (illustrative)

Goa offers **golden sandy beaches** and lively nightlife.



DESCRIPTION B (illustrative)

Visitors love the **golden sandy beaches** of coastal Goa.

Shared Text · Alignment Score

Dynamic Programming Features



BITMASK DP

Multi-destination route ordering



INTERVAL DP

Choosing non-overlapping activities



TREE DP

Hierarchical travel-plan optimization under a budget



SOS DP

Preference-based recommendations

ALSO DP TECHNIQUES

Edit Distance · Needleman–Wunsch · Smith–Waterman

Used for typo suggestions and description alignment.

Multi-Destination Itinerary Optimization

Selected Destinations

↓

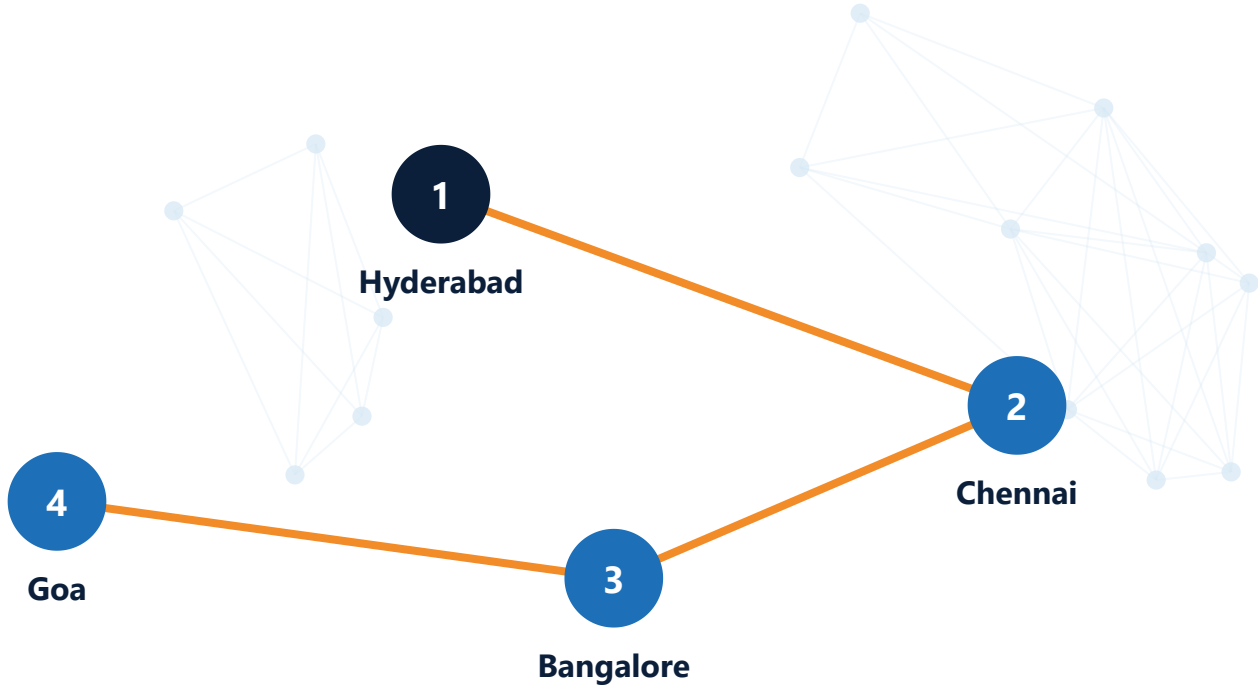
OSRM Route Matrix

↓

Bitmask DP

↓

Ordered Itinerary
+ Route Totals



Illustrative example: Hyderabad → Chennai → Bangalore → Goa

Route costs come from OSRM; Bitmask DP selects an optimized visiting order.

Illustrative layout only – not an actual computed result.

SOS DP – Travel Preference Matching

8 PREFERENCE CATEGORIES (8-BIT MASK)

Beach	00000001	Food	00010000
Historical	00000010	Nightlife	00100000
Adventure	00000100	Relaxation	01000000
Nature	00001000	Shopping	10000000

ILLUSTRATIVE EXAMPLE

Beach + Nightlife + Relaxation

```
00000001 + 00100000 + 01000000
= 01100001
```

Bits 0, 5 and 6 are set (1 + 32 + 64 = 97).

Preference Mask → **SOS DP (subset sums over 2^8 masks)** → **Ranked Recommendations**

Algorithm Testing



23 AUTOMATED JAVA TESTS PASSED

0 failures

Full backend suite: 14 DSA tests plus 9 flow/API tests.

Naïve Search	✓	LCP	✓
KMP	✓	Needleman–Wunsch	✓
Z-Function	✓	Smith–Waterman	✓
Rabin–Karp	✓	Bitmask DP	✓
Aho–Corasick	✓	Interval DP	✓
Edit Distance	✓	Tree DP	✓
Suffix Array	✓	SOS DP	✓

Edmonds–Karp · Maximum-flow activity allocation ✓

Algorithmic Complexity

Algorithm	Time Complexity
Naïve Search	O(nm)
KMP	O(n + m)
Z-Function	O(n + m)
Rabin–Karp	avg O(n + m); worst O(nm)
Aho–Corasick	O(total pattern length + n + matches)
Edit Distance	O(nm)
Suffix Array	$O(n \log^2 n)$

Algorithm	Time Complexity
LCP (Kasai)	O(n)
Needleman–Wunsch	O(nm)
Smith–Waterman	O(nm)
Bitmask DP	$O(n^2 \cdot 2^n)$
Interval DP (scheduling)	O(n log n)
Tree DP	$O(N \cdot B^2)$ time; $O(N \cdot B)$ space
SOS DP	$O(k \cdot 2^k)$

Variables: n, m = text/pattern (or compared string) lengths; n alone = Suffix Array/LCP string length. Bitmask: n destinations; Interval: n activities. Tree: N nodes, B budget. SOS: k preference bits. Network Flow: $O(V \cdot E^2)$, where V = vertices and E = edges.

Network Flow: Activity Allocation

TravelSync uses maximum flow to assign travelers to activities they are interested in while respecting available seats.

SOURCE
Start of the network

→

TRAVELERS
One node per person
Source edge capacity = 1

→

ELIGIBLE ACTIVITIES
Connect only matching interests
Each match edge capacity = 1

→

SINK
Activity seat limits
Capacity = available seats

1 · SEARCH
Breadth-first search finds a shortest augmenting path in the residual graph.

2 · AUGMENT
Send flow along the path. Reverse residual edges can reroute earlier assignments.

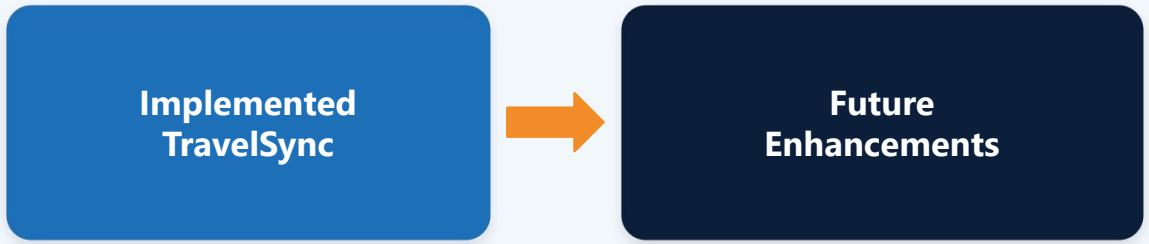
3 · MAXIMIZE
Stop when no path remains. The flow value is the maximum travelers assigned.

Unassigned travelers are shown in the planner. Edmonds–Karp complexity: $O(V \cdot E^2)$, with V vertices and E edges. Evaluated by backend engine, service and API tests.

Conclusion & Future Scope

CONCLUSION

TravelSync combines a React web interface, external travel-data services and a Java Spring Boot backend with DSA-powered features for search, text comparison, route ordering, activity scheduling, capacity-constrained activity allocation and recommendations.

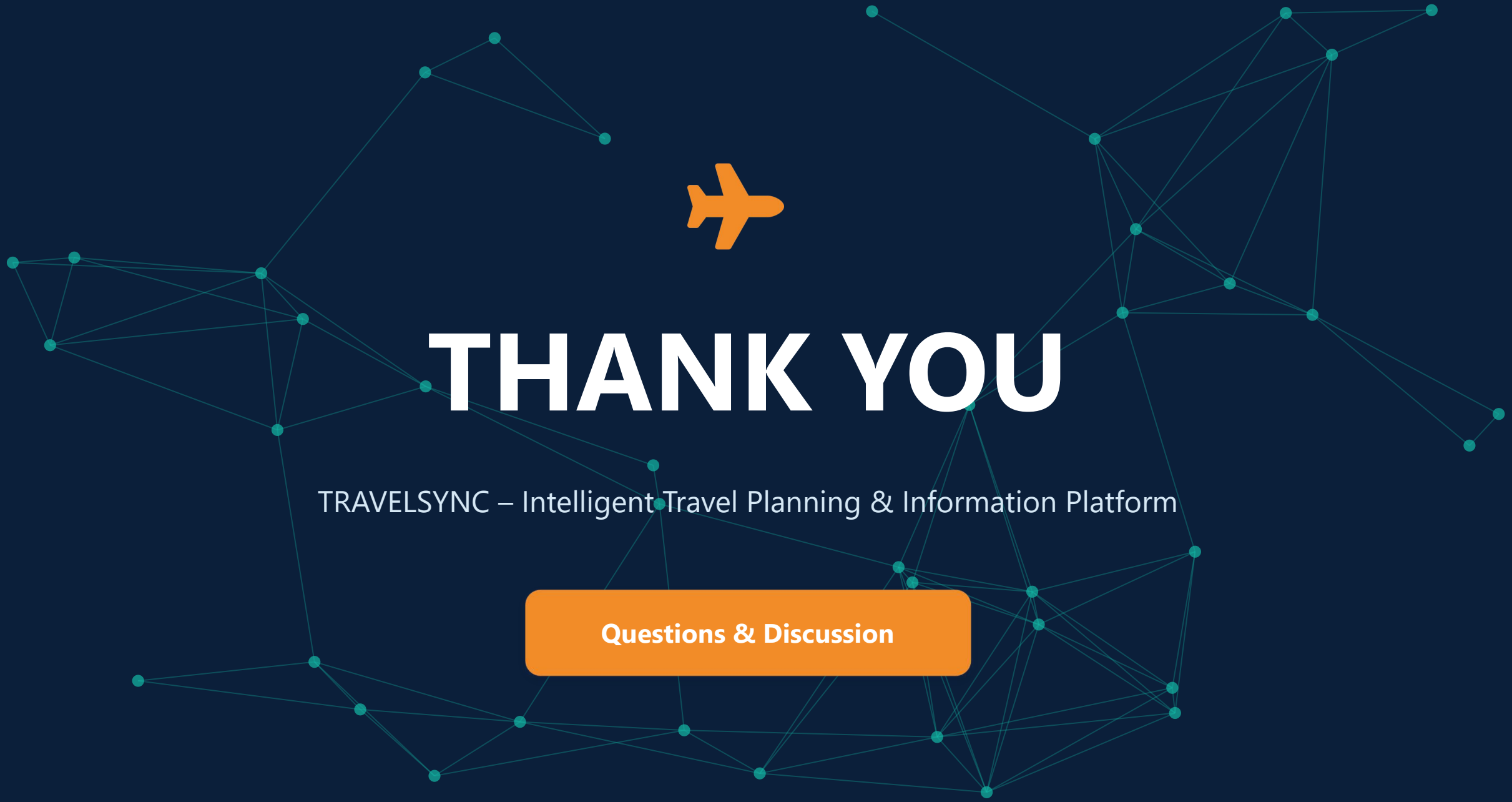


IMPLEMENTED

- **React + Vite web interface**
- **External API integrations**
- **Preference-based recommendations**
- **15 DSA engines · 23 passing Java tests**

FUTURE SCOPE

- **Persistent database storage**
- **Broader live datasets**
- **More robust recommendation personalization**
- **Deployment**
- **Additional real-world performance evaluation**



THANK YOU

TRAVELSYNC – Intelligent Travel Planning & Information Platform

Questions & Discussion